

CSCE 606 SOFTWARE ENGINEERING · MODULE 12 · LECTURE NOTES

Process

How teams organise work, and what the evidence says.

MODULE OUTCOMES

- Explain why waterfall persisted and what it got right.
- Explain what the Agile Manifesto actually said.
- Distinguish agile principles from Agile ceremony.
- Estimate honestly, and explain why estimates are unreliable.
- Judge a process by what it optimises.

1 Waterfall, and what it actually said

ROYCE ARGUED AGAINST THE MODEL HE IS CREDITED WITH

Winston Royce's 1970 paper is universally cited as the origin of the waterfall model. He presented the sequential diagram — requirements, design, implementation, verification, operations — and then wrote that implementing it this way 'is risky and invites failure'.

His actual recommendation was to **build it twice** — a pilot version to learn from — and to feed testing results back into design. That is an iterative process with feedback loops.

The diagram was adopted and the argument accompanying it was not. It is among the more consequential misreadings in the field's history, and it shaped a generation of government procurement.

What the sequential model got right remains right: thinking before building is better than not; writing decisions down preserves them; and a change costs more the later it is made. None of this is in dispute.

The genuine failure is **feedback latency**. Requirements cannot be fully known in advance (Module 02); a misunderstanding in week two surfaces in month nine; there is no working software to inspect, so progress can only be reported rather than verified; and change is framed as a failure of planning, so it is resisted rather than accommodated.

2 What the Agile Manifesto said

Seventeen people, 2001, four sentences. Worth reading in the original, because most of what is done in its name is not in it.

We value	over
Individuals and interactions	processes and tools
Working software	comprehensive documentation
Customer collaboration	contract negotiation
Responding to change	following a plan

The document then states explicitly: *'That is, while there is value in the items on the right, we value the items on the left more.'* That caveat is almost always omitted in practice, which is how 'agile means we do not write documentation' became a widespread and entirely unsupported reading.

PRINCIPLES AND CEREMONY ARE NOT THE SAME THING

The principles are modest and largely well supported: deliver working software frequently; welcome changing requirements; get feedback from real users; keep the design simple; have the team reflect and adjust at intervals.

The ceremony is what was subsequently sold: two-week sprints, story points, velocity tracking, daily stand-ups, sprint planning poker, retrospectives with sticky notes, certification programmes, and consultants to install all of it.

Much of the ceremony has no evidential support, and some of it contradicts the principles it claims to implement — a sprint commitment is a plan you have agreed not to change for two weeks, which is the fourth value inverted.

The useful test for any practice: does it shorten the feedback loop? Daily stand-ups can, if they surface blockers. They do not, if they are status reports to a manager. The same ritual, opposite effects — which is why arguing about the ritual rather than its function is unproductive.

3 Estimation

Software estimates are unreliable, and the reasons are structural rather than a matter of insufficient discipline.

- 1 **The planning fallacy.** People systematically underestimate task duration, and continue to do so after being shown evidence that they do. This is a robust finding well outside software.
- 2 **Unknown unknowns.** The work you did not know would be necessary is by definition absent from the estimate, and on novel work it is frequently the majority.
- 3 **Estimates become commitments.** The moment a number is written down it is treated as a promise, so estimators pad, so the numbers carry less information, so trust declines, so more padding is required. The feedback loop is self-defeating.
- 4 **Estimating requires understanding the work,** and understanding the work generally arrives by doing it. The estimate is required at the moment of minimum information.

Approach	What it does
Historical throughput	Measure how long comparable items have actually taken on this team, with this codebase. Beats expert judgement consistently, and almost nobody collects the data.
Ranges rather than points	'Two to six weeks, most likely three.' Honest about uncertainty, and harder to convert into a commitment.
Reference class forecasting	Compare against the distribution of similar completed projects rather than reasoning from the inside. Corrects the planning fallacy more effectively than trying harder.
Slice smaller	Small items estimate better, and errors on many small items partially cancel rather than compounding.
#NoEstimates	Slice work into items of roughly uniform size, count how many are completed per week, and forecast from the count. Stronger than it sounds once items are genuinely uniform.

MEASURE CYCLE TIME

Cycle time is the elapsed time from starting work on an item to delivering it. It is measured rather than predicted.

Plot the distribution and you have a forecast with evidence behind it: 'items of this type take three to ten days, with 85% completing within eight.' That is derived from this team working on this codebase, which is exactly the population the question is about.

It improves automatically as data accumulates, requires no estimation meetings, and cannot be padded or negotiated — it is a record of what happened.

It also directs attention usefully. Reducing cycle time means removing *waiting* — for review, for CI, for deployment, for a decision — and in most teams waiting dominates. Almost none of cycle time is typing, which is the activity estimation focuses on.

4 What the evidence supports

Practice	Evidence	Source
Small batch sizes and frequent delivery.	Strong	DORA, consistently across years and organisation types.
Version control, CI, automated deployment, test automation.	Strong	DORA.
Loosely coupled architecture allowing independent deployment.	Strong	DORA. Module 03's argument, measured.
Psychological safety.	Strong	Google's Project Aristotle; DORA's culture findings. The strongest single predictor identified.
Heavyweight change approval boards.	Negatively correlated	DORA. Associated with worse outcomes on both speed and stability, which is the opposite of their purpose.
Specific ceremonies: sprint length, story points, planning poker.	Weak to none	Little rigorous study; what exists is inconclusive.

THE STRONGEST PREDICTOR IS WHETHER PEOPLE CAN SPEAK UP

Google's Project Aristotle studied 180 teams looking for the determinants of effectiveness. The strongest factor was not individual skill, seniority, team size, or process.

It was **psychological safety**: whether team members believe they can admit a mistake, ask a question that might sound ignorant, challenge a decision, or report bad news without being penalised.

The mechanism is directly relevant to engineering. In a safe team, problems surface early, bad news travels fast, people ask rather than guess, bugs are reported rather than concealed, and bad estimates are corrected rather than defended.

Every one of those is a *feedback loop*, which this module has identified as the thing that matters. That makes blameless culture an engineering practice rather than a management nicety: it determines how quickly you find out that something is wrong.

5 Judging a process

- 1 **Does it shorten the feedback loop?** The single most useful question, and most practices can be assessed against it directly.
- 2 **Does it surface problems early,** or does it defer their discovery to a deadline?
- 3 **Does it reduce batch size?** Smaller batches are the best-supported finding in the field.
- 4 **Does it make work visible?** Invisible work cannot be prioritised, balanced, or improved.
- 5 **Is anyone actively deciding to continue it?** Or is it simply what has always been done?

That last question identifies most process dysfunction. Practices accumulate, the reasons for them expire, and nobody removes them because nobody is responsible for the question. A practice that nobody can justify is ceremony. **Stop it for a month and observe what breaks** — frequently nothing does, and when something does, you have learned what the practice was actually for, which is more than anyone knew before.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Royce — Managing the Development of Large Software Systems (1970, free)	The waterfall paper. Read sections 3 onward, where he argues against the model he had just drawn.
The Agile Manifesto and its twelve principles	Four sentences and twelve principles. Takes five minutes and corrects a great deal.
Forsgren, Humble & Kim — Accelerate / DORA	The empirical research. Read the capability pages rather than only the four metrics.
Google re:Work — Project Aristotle	The psychological safety finding, with the methodology.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read Royce's paper and identify the passage where he recommends against the sequential model. Write a paragraph on how the misreading happened.
- 2 Read the twelve Agile principles and identify which practices in a team you know follow from them and which do not.
- 3 Measure cycle time for the last twenty items in a project you can observe. Plot the distribution and produce an 85th-percentile forecast.
- 4 Compare that forecast against the original estimates for the same items. Quantify the estimation error and its direction.
- 5 List every recurring process practice in a team you know. For each, state what feedback loop it shortens. Identify the ones with no answer.
- 6 Break down cycle time into working time and waiting time for five items. Report the ratio.
- 7 Find a team practice that has been retained past its original purpose. Propose stopping it and predict what will break.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What did Royce actually recommend, and what did waterfall get right?
- 2 What is the core problem with sequential development?
- 3 State the four values of the Agile Manifesto and the caveat that accompanies them.
- 4 Distinguish agile principles from Agile ceremony, and give the test for any practice.
- 5 Give four structural reasons software estimates are unreliable.
- 6 What is cycle time, and why is it better than estimation?
- 7 What is the strongest predictor of team effectiveness found, and why does it matter for engineering specifically?